



Chương 5: Nền tảng của Học máy (Fundamentals of ML)

Học Sâu (Deep Learning)

Nội dung chương

1. Tối ưu hóa
(Optimization)
và Khái quát hóa
(Generalization)

2. Tác động của Dữ liệu Nhiễu
(Noisy Data)

3. Các phương pháp Đánh giá Mô hình

4. Dung lượng Mô hình
(Model Capacity)

5. Các Phương pháp
Regularization

6. Giả thuyết Đa tạp (The Manifold Hypothesis)

7. Tổng kết

Sự giằng co cốt lõi trong Học máy



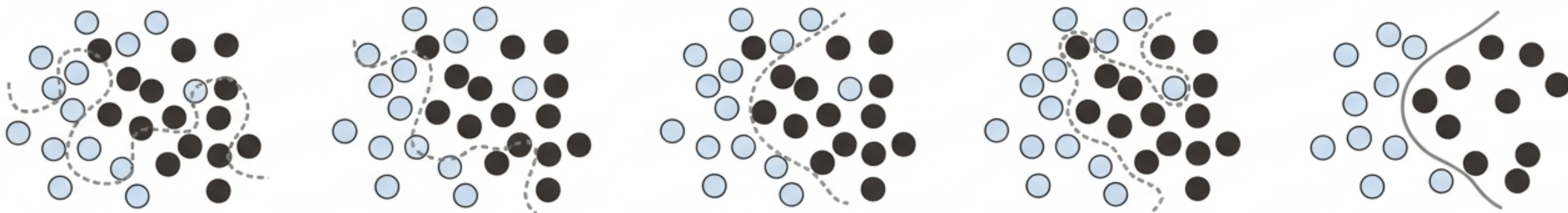
Tối ưu hóa (Optimization): Quá trình điều chỉnh mô hình để đạt hiệu suất tốt nhất trên tập dữ liệu huấn luyện (Training data).



Khái quát hóa (Generalization): Khả năng mô hình dự đoán chính xác trên dữ liệu hoàn toàn mới chưa từng thấy (Test data).

Mâu thuẫn: Mục tiêu là đạt Generalization cao nhất. Tuy nhiên, Gradient Descent chỉ tác động trực tiếp vào Optimization. Sau một thời gian, Optimization càng tốt thì Generalization lại càng đi xuống.

Hiện tượng Underfitting và Overfitting



Underfitting



Robust Fit



Overfitting



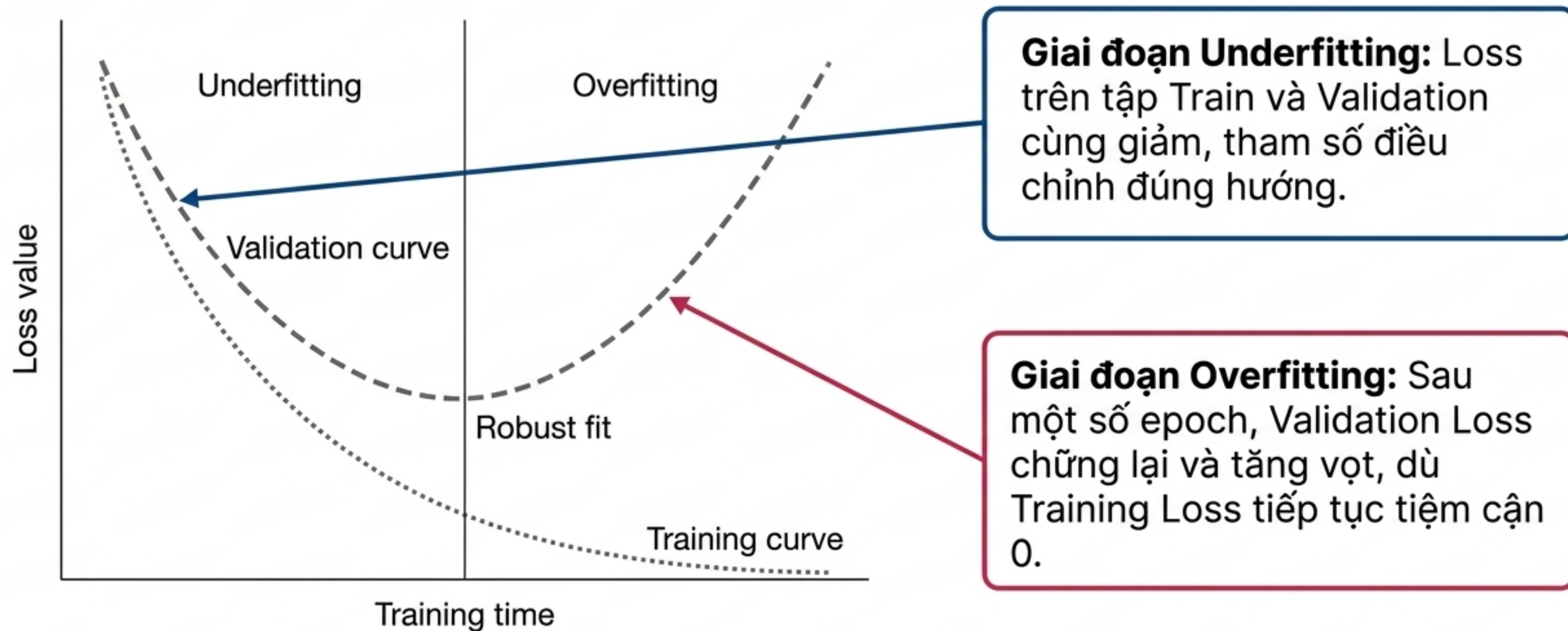
Underfitting (Chưa khớp): Mô hình quá đơn giản, không học được quy luật trong dữ liệu. (Vd: Dùng đường thẳng tuyến tính khớp dữ liệu parabol).

Robust Fit (Khớp chuẩn): Mô hình học được xu hướng thực sự của dữ liệu.

Overfitting (Quá khớp): Mô hình quá phức tạp, học thuộc lòng cả “nhiều” (noise), làm đường biên quyết định cong queo, mất tính tổng quát.

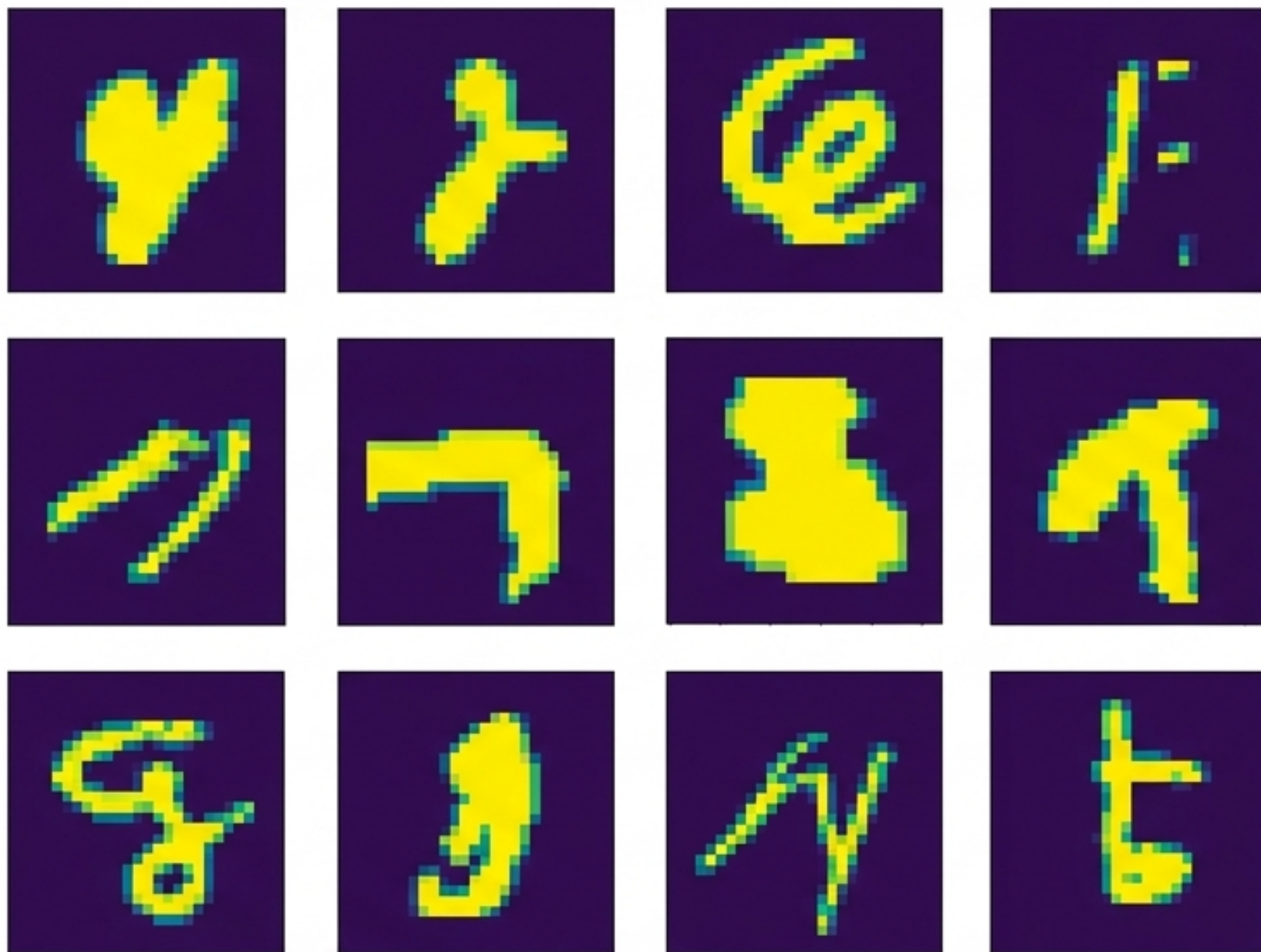
Hình 5.1: Mô phỏng Underfitting, Robust Fit và Overfitting

Vòng đời điển hình của mô hình (Universal Overfitting)



Hình 5.2: Hành vi Overfitting kinh điển

Dữ liệu không hợp lệ (Invalid Inputs)



Trong thực tế, dữ liệu chứa nhiều tạp âm, nhòe nhòe inoẹt. Ví dụ: MNIST có ảnh lỗi, mờ đến mức mắt người cũng không nhận diện được.



Nếu mạng có đủ **dung lượng**, nó sẽ “**học vẹt**” những hình nhiễu này, lãng phí tài nguyên ghi nhớ chúng → Suy giảm tính khái quát.

Hình 5.3: Các ảnh bị nhiễu nặng trong tập MNIST

Dữ liệu bị gán nhãn sai (Mislabelled Data)

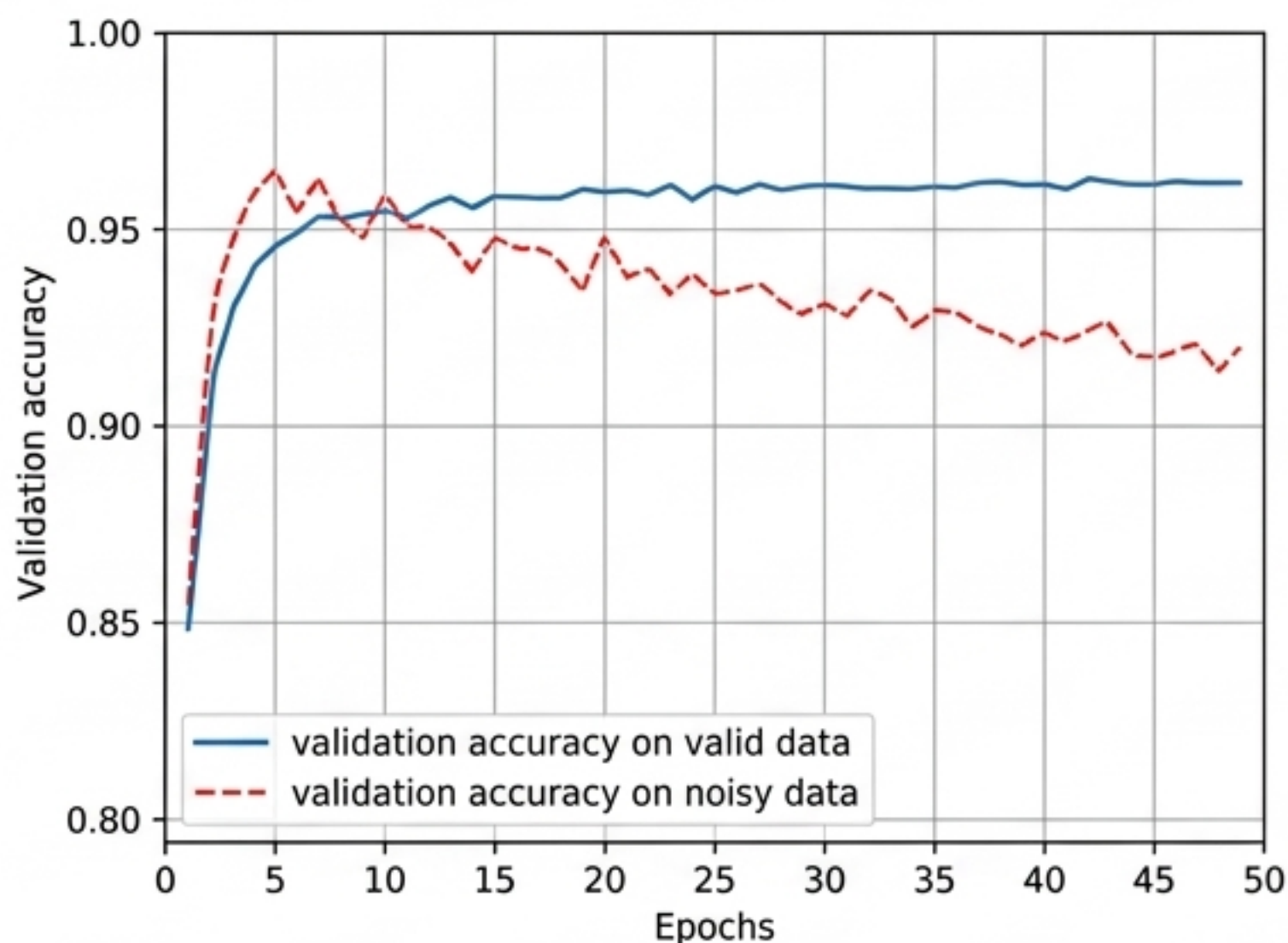


Do sai sót chủ quan của con người (Human Error)

Dữ liệu có thể bị gán nhãn sai hoàn toàn (Vd: số 4 bị gán nhãn là số 9).

Mô hình cố uốn cong ranh giới quyết định để "chữa" nhãn sai này → Dẫn đến nhận diện sai hàng loạt ảnh tương tự trong tương lai.

Sự phình to của Kênh Nhiễu (Noise Channels)



Hình 5.5: Tác động của kênh nhiễu đến Accuracy



Thí nghiệm: Thêm ma trận toàn nhiễu trắng (white noise) hoặc toàn số 0 vào dữ liệu MNIST.

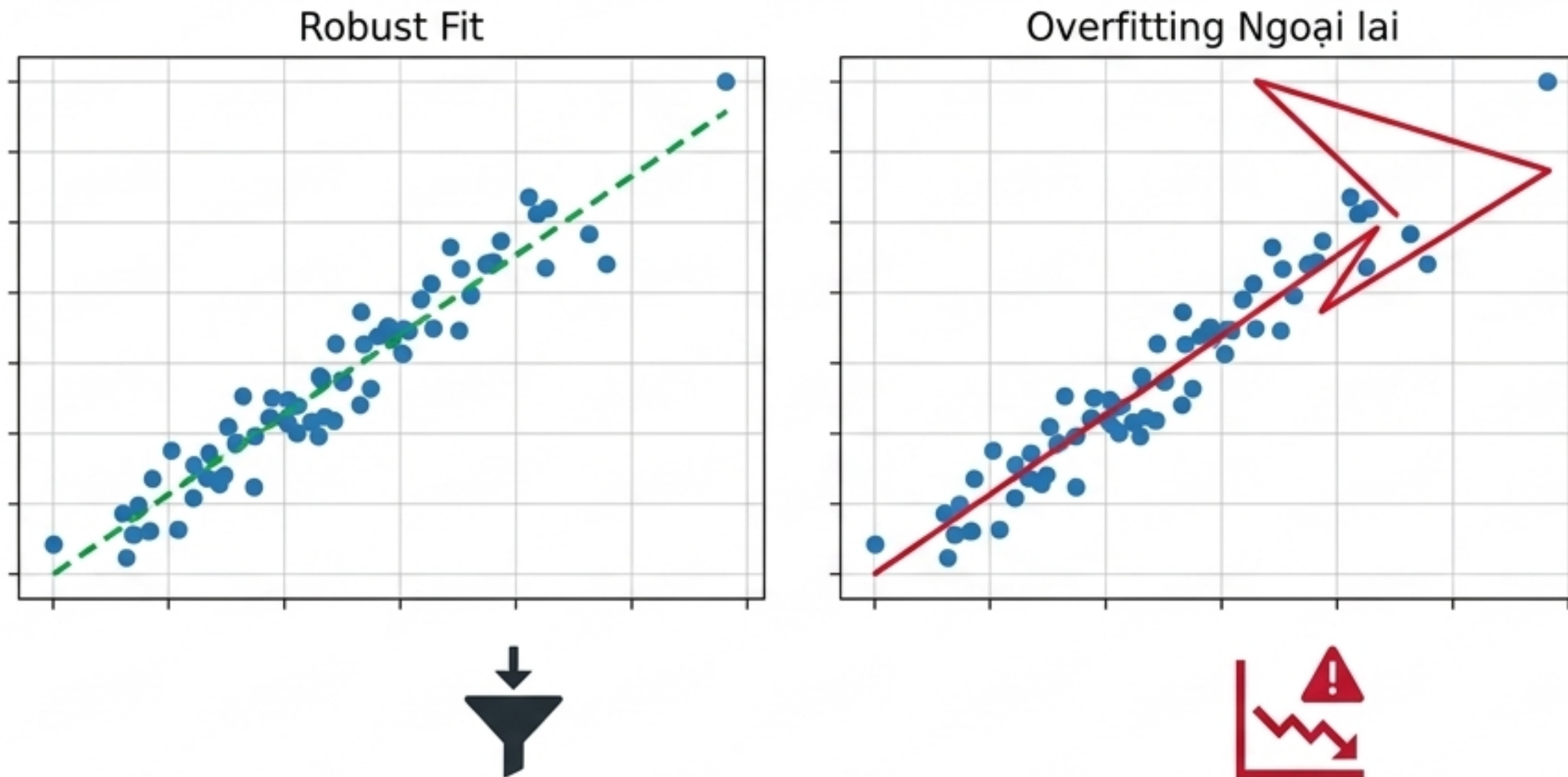


Kết quả: Kể cả khi dữ liệu MNIST hợp lệ nguyên vẹn, mạng bị phân tâm. Validation Accuracy giảm rõ rệt.



Lý do: Mô hình vô tình tìm ra các mẫu (patterns) giả mạo trong tập nhiễu và sử dụng chúng để ra quyết định → Suy giảm Generalization.

Ngoại lai (Outliers) và Sự quá khớp



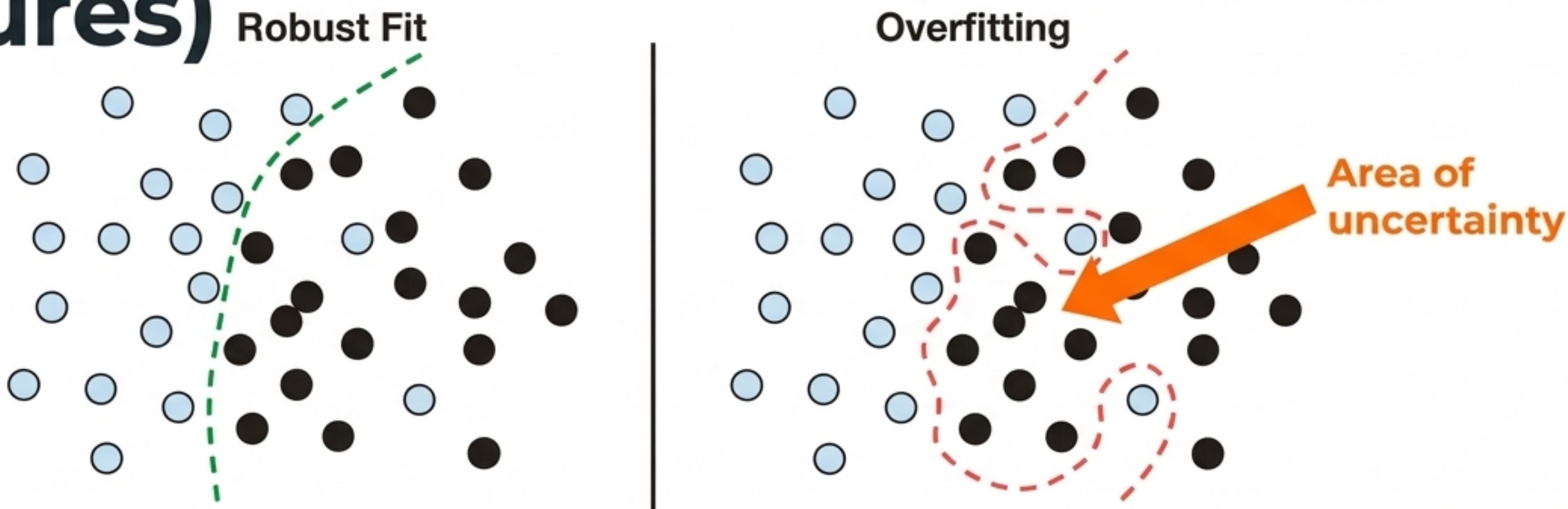
Điểm nhiễu thường là "**Ngoại lai**" (Outliers) - dữ liệu lạc lõng giữa quần thể lớp khác.

Gradient Descent nỗ lực cực độ để phân loại đúng 100% tập huấn luyện, ép ranh giới quyết định vươn xa ra ngoài bọc lấy outlier.

Hậu quả: Ranh giới quyết định trở nên **góc cạnh** thay vì là **đường cong mượt mà** bao quát.

Hình 5.6: Robust fit (trái) vs. Overfitting ngoại lai (phải)

Đặc trưng mơ hồ (Ambiguous features)



Vùng bất định (Uncertainty):
Vùng giao thoa nơi dữ liệu có
xác suất 50% lớp A và 50%
lớp B.

Vấn đề: Overfitting vạch
đường biên răng cưa uốn éo
để cố phân chia tách - thay vì
dùng đường thẳng thống kê.

Khắc phục: Phải cho phép
mô hình phạm lỗi trên tập
Train (Training Loss không
bao giờ bằng 0).

Hình 5.7: Overfitting trên vùng dữ liệu bất định

Tại sao phải chia 3 tập dữ liệu?

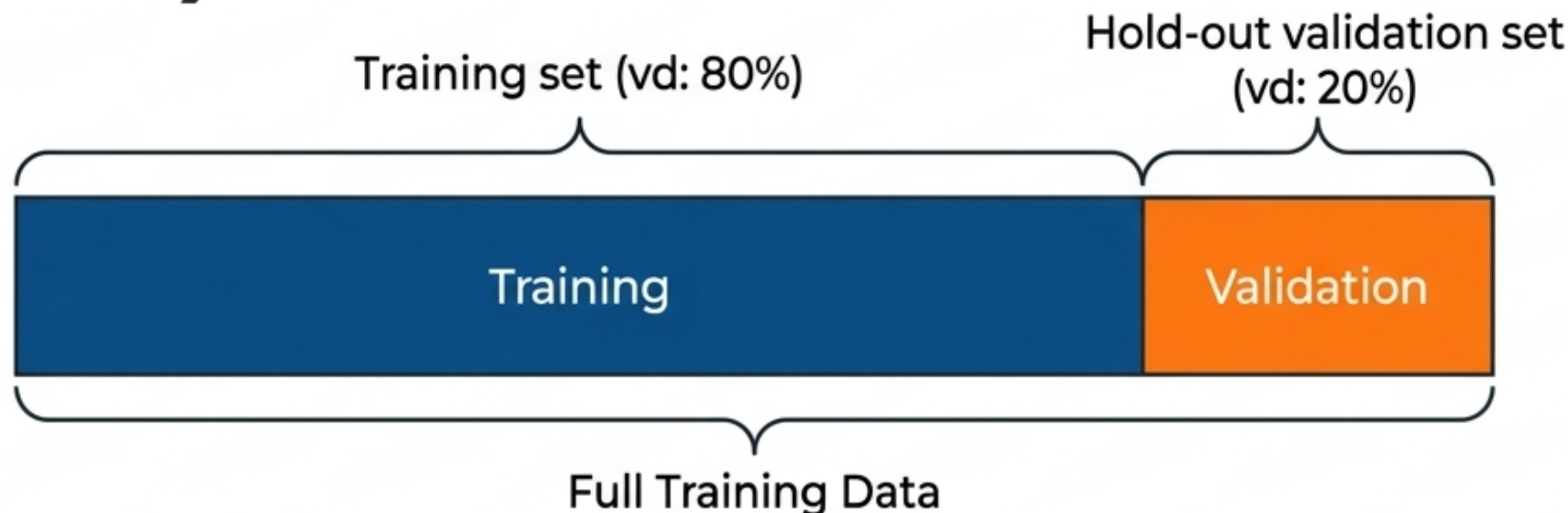


Huấn luyện không chỉ chỉnh "Tham số" (Weights/Biases) mà còn "Siêu tham số" (Hyperparameters - số lớp, learning rate) dựa trên điểm Validation.

Information Leak: Dùng Validation liên tục làm la bàn khiến mô hình gián tiếp "học lỏm" cấu trúc tập này → Overfit trên chính Validation.

Giải pháp: Bắt buộc cần tập thứ 3 (Tập Test) - Hoàn toàn mới, chỉ chạy đúng 1 lần trước khi ứng dụng thực tế.

Tách tập Validation đơn giản (Hold-out Validation)



Quy trình: Xáo trộn (Shuffle) và tách một phần cố định (vd: 20%) từ tập Training.



Ưu điểm:

Rất nhanh gọn, phù hợp khi khối lượng dữ liệu khổng lồ.



Nhược điểm:

Nếu dữ liệu ít, tập Validation không mang tính đại diện thống kê, sai số đánh giá lớn khi đổi seed.

Hình 5.8: Quy trình Hold-out Validation

Mã nguồn: Hold-out Validation

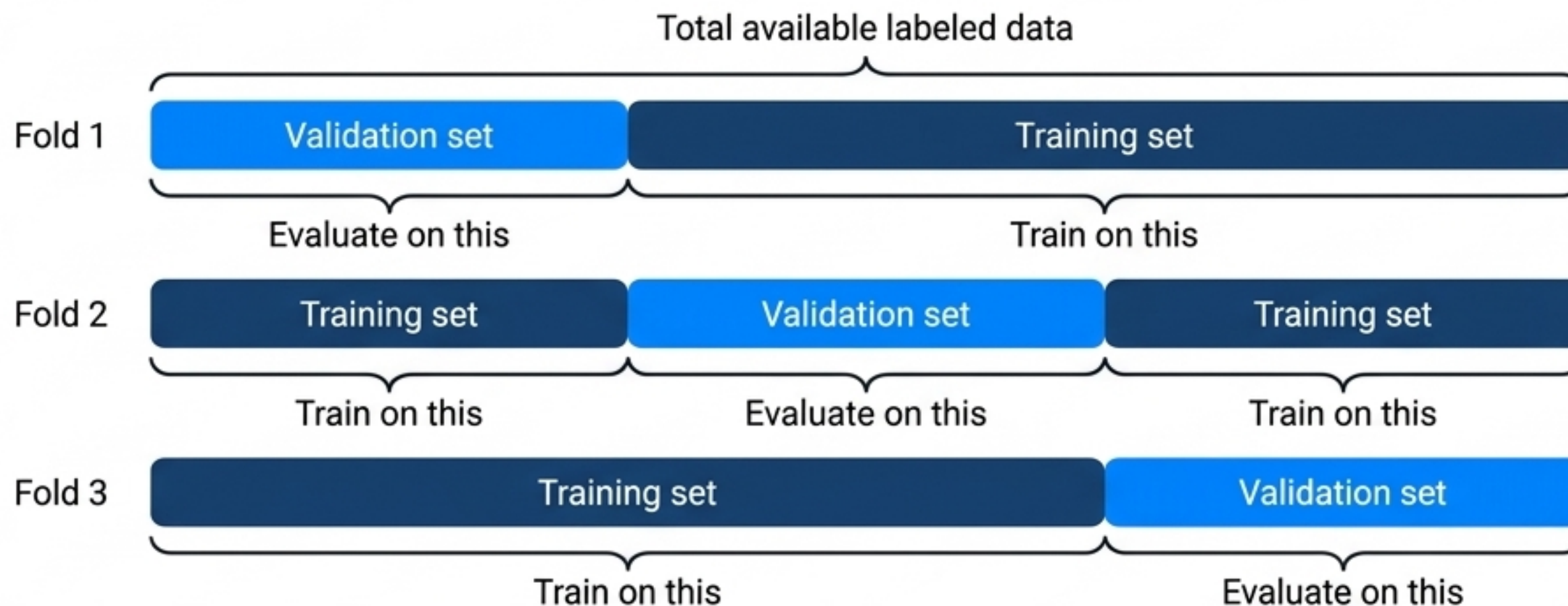
```
1 import numpy as np
2
3 num_val_samples = 10000
4 np.random.shuffle(data) # Xáo trộn ngẫu nhiên
5
6 # Tách 10k mẫu làm Validation
7 val_data = data[: num_val_samples]
8 train_data = data[num_val_samples :]
9
10 model = get_model ()
11 model.fit(train_data , epochs =20)
12 val_score = model.evaluate(val_data)
13
14
15 # Sau khi điều chỉnh hyperparameter thành công ,
16 # ta huấn luyện lại 1 mô hình tren toàn bộ Train + Validation
17 model = get_model ()
18 model.fit(np.concatenate ([train_data , val_data ]), epochs =20)
19 test_score = model.evaluate(test_data)
20
```

Xáo trộn ngẫu nhiên dữ liệu trước khi cắt.

Đánh giá bằng tập Validation riêng biệt.

Huấn luyện lại mô hình chốt trên toàn bộ Train + Validation.

Đánh giá chéo K-Lần (K-Fold Validation)



Quy trình: Giải quyết vấn đề dữ liệu thừa thớt bằng cách chia dữ liệu thành K phần.

Lặp K vòng: lấy 1 Fold làm Validation, K-1 Fold làm Training.

Điểm cuối là trung bình K điểm số.



Ưu điểm: Tận dụng tối đa dữ liệu, độ chính xác đánh giá cao.



Nhược điểm: Chi phí tài nguyên tính toán gấp K lần.

Hình 5.9: Quy trình K-Fold Validation (với K=3)

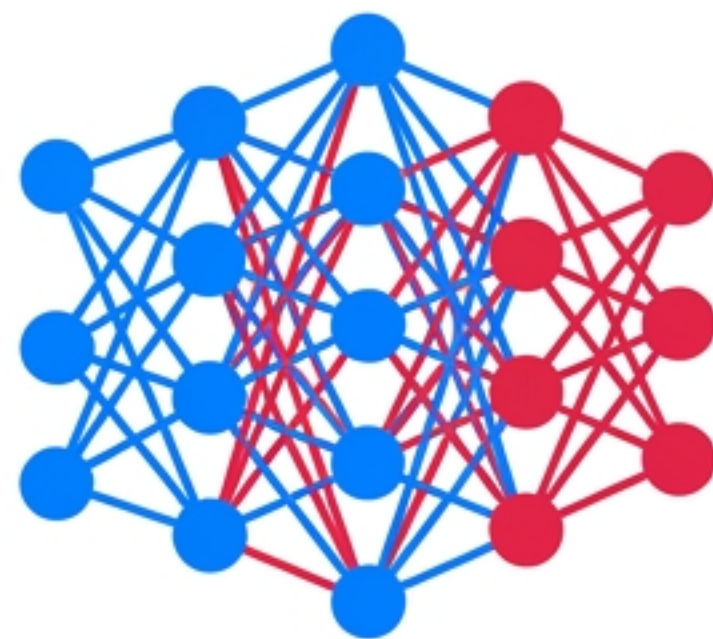
Mã nguồn: K-Fold Validation

```
1 k = 4
2 num_val_samples = len(data) // k
3 val_scores = []
4
5 for fold in range(k):
6     # Lay ra phan Validation rieng
7     val_data = data[num_val_samples * fold : num_val_samples * (fold + 1)]
8
9     # Gop phan dau va cuoi de tao thanh tap Train
10    train_data = np.concatenate(
11        [data[: num_val_samples * fold], data[num_val_samples * (fold + 1) :]], axis=0
12    )
13
14    model = get_model () # Khoi tao lai mo hinh moi hoàn toàn!
15    model.fit(train_data , epochs =100, batch_size =16)
16    score = model.evaluate(val_data)
17    val_scores.append(score)
18
19 average_val_score = np.mean(val_scores) # Lay trung binh
```

Vòng lặp K lần huấn luyện hoàn toàn độc lập.

Ghép nối các đoạn dữ liệu đầu/cuối để tạo tập Train.

Điểm đánh giá cuối cùng là trung bình (mean) của toàn bộ K vòng.



Giới thiệu bậc đại học môn Máy học

Giới thiệu bậc đại học môn Máy học

Chương 5 - Phần 2: Tối ưu hóa Mô hình & Kỹ thuật Regularization

KHÔNG GIAN THIẾT KẾ MÔ HÌNH: TỪ DUNG LƯỢNG ĐẾN KỸ THUẬT TÍNH NĂNG

Xử lý khi Validation Loss không chịu giảm



- **Triệu chứng:** Loss không giảm, Accuracy kẹt ở mức chọn bừa (Random Guess).
- **Nguyên nhân:** Learning Rate quá khổng lồ (ví dụ: 1.0), khiến bước Gradient bay nhảy hỗn loạn.

- **Giải pháp:** Hạ Learning Rate (ví dụ: $1e-2$) để giảm trơn tru. Tăng Batch Size để cung cấp định hướng đạo hàm chính xác, ít nhiễu.

```
model.compile(  
    optimizer=keras.optimizers.RMSprop(  
        learning_rate=1e-2),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"],  
)  
model.fit(train_images, train_labels,  
    epochs=10, batch_size=128)
```

Dung lượng Mô hình (Model Capacity) là gì?

Quá Nhỏ (Underfitting)



Không đủ khả năng 'ghi nhớ' quy luật phi tuyến tính phức tạp.

Lý Tưởng (Mục Tiêu)



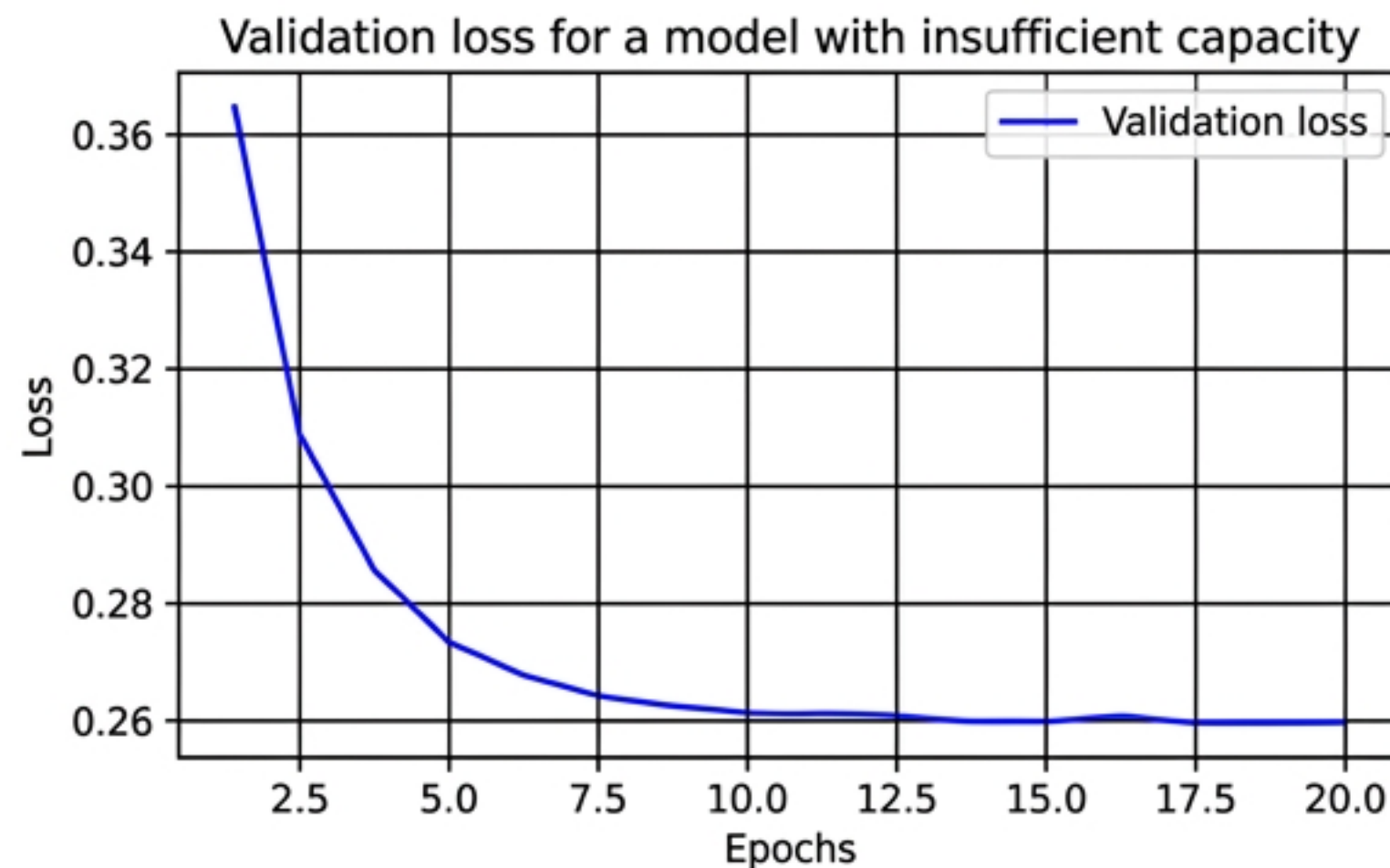
Giảm loss tối đa nhưng trì hoãn việc overfit muộn nhất. Vừa khít.

Quá Lớn (Overfitting)



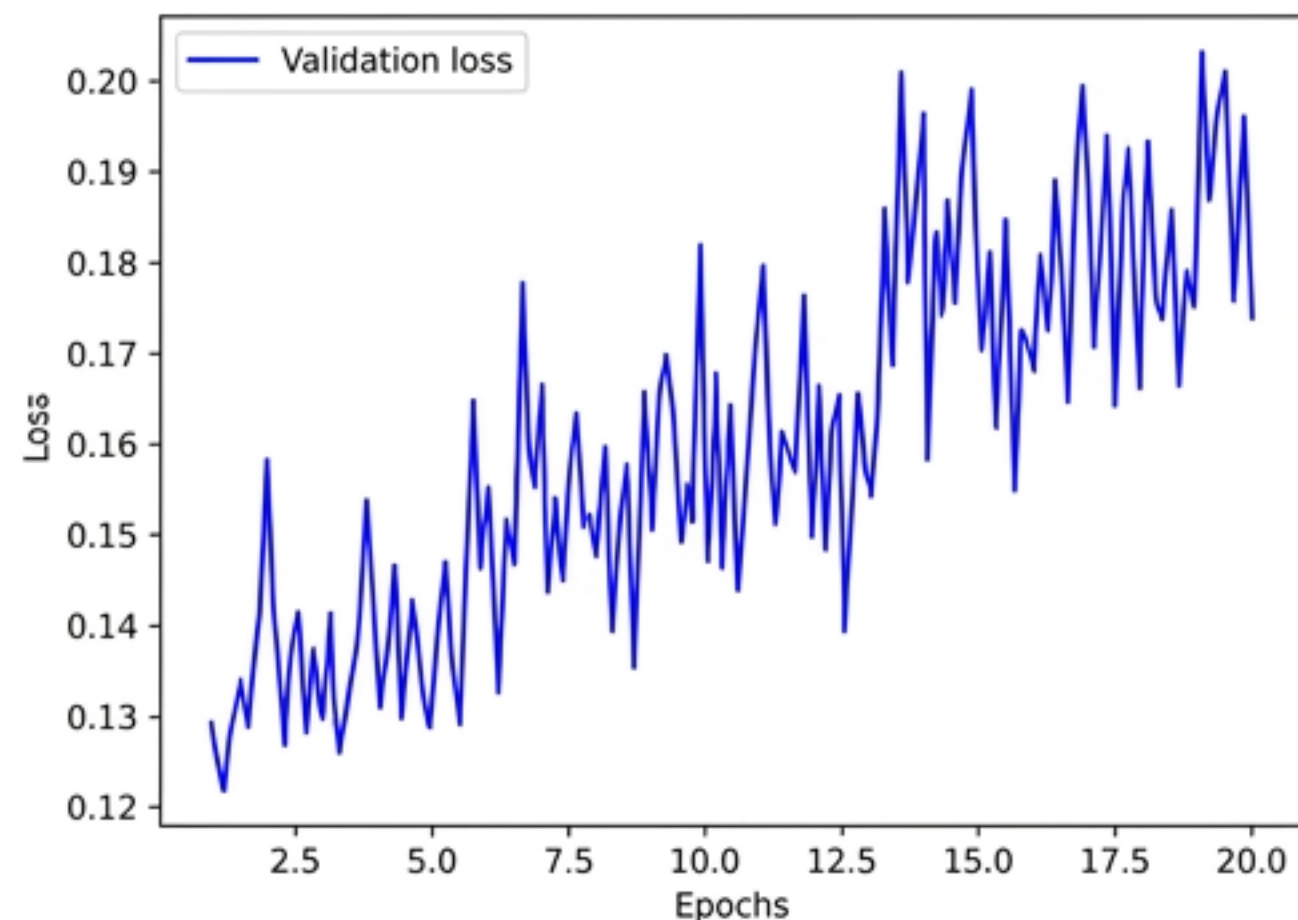
Dư thừa, học vẹt cực nhanh (mapping trực tiếp input ra output).

Khi Dung lượng Mô hình Không Đủ



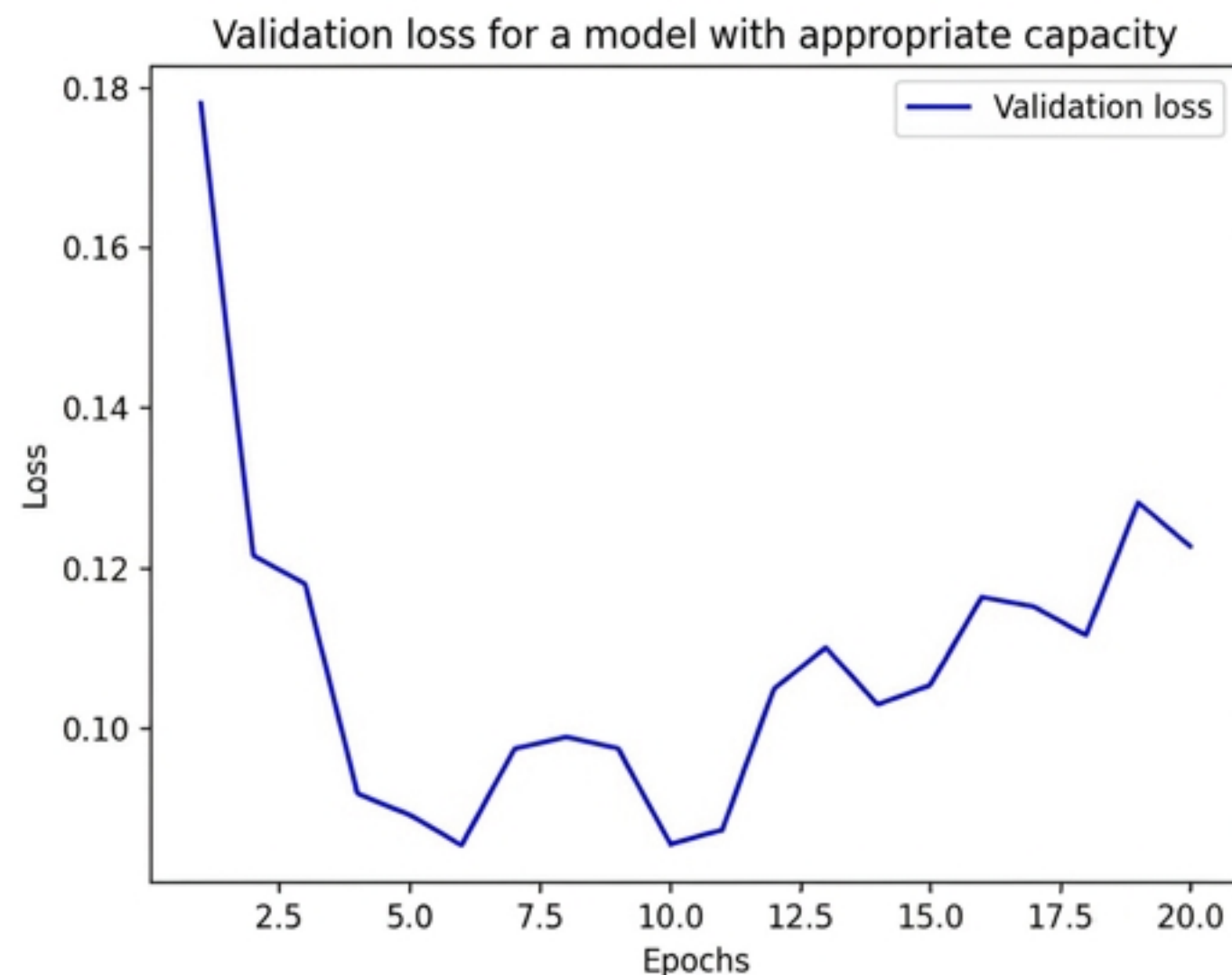
- Thử nghiệm: Mạng hồi quy Logistic siêu nông trên MNIST.
- Triệu chứng: Validation Loss chững lại ở mức 0.26 và giữ nguyên (Stalled).
- Kết luận: Mô hình không đủ sức mạnh để khái quát không gian giả thuyết phức tạp.

Khi Dung lượng Mô hình Quá Lớn



- **Thử nghiệm:** Mạng khổng lồ (3 lớp x 2048 nơ-ron).
- **Triệu chứng:** Ghi nhớ thần tốc. Nhớ toàn bộ tập Training trong vài giây.
- **Kết quả:** Validation Loss chạm đáy ngay tại Epoch 1, sau đó vọt lên mạnh mẽ với biên độ siêu nhiều (noise).

Dung lượng Mô hình Lý Tưởng



- Thiết kế: 2 lớp ẩn (mỗi lớp 128 units).
- Kết quả: Validation Loss giảm từ từ và duy trì ở mức tối thiểu, chỉ tăng nhẹ ở epoch thứ 8.
- Đánh giá: Kiến trúc cân bằng tuyệt vời giữa tối ưu hóa và khái quát hóa.

So sánh Dung lượng: Bài toán IMDB



Mạng Gốc

```
layers.Dense(16, activation="relu")  
layers.Dense(16, activation="relu")
```

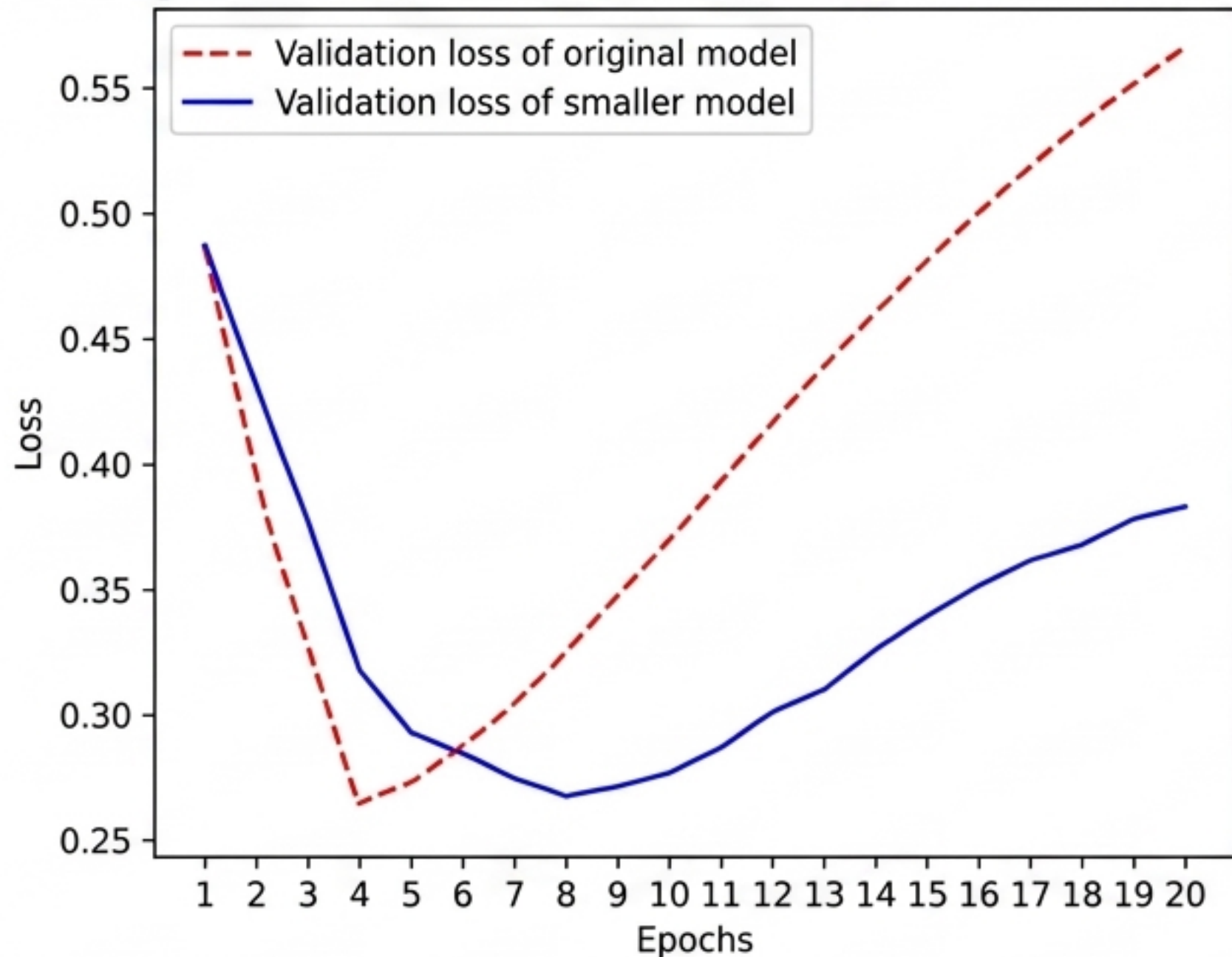
Mạng Nhỏ

```
layers.Dense(4, activation="relu")  
layers.Dense(4, activation="relu")
```

Thử nghiệm đẩy kiến trúc từ 16 units xuống 4 units (để giảm thiểu Overfitting), và vọt lên 512 units.

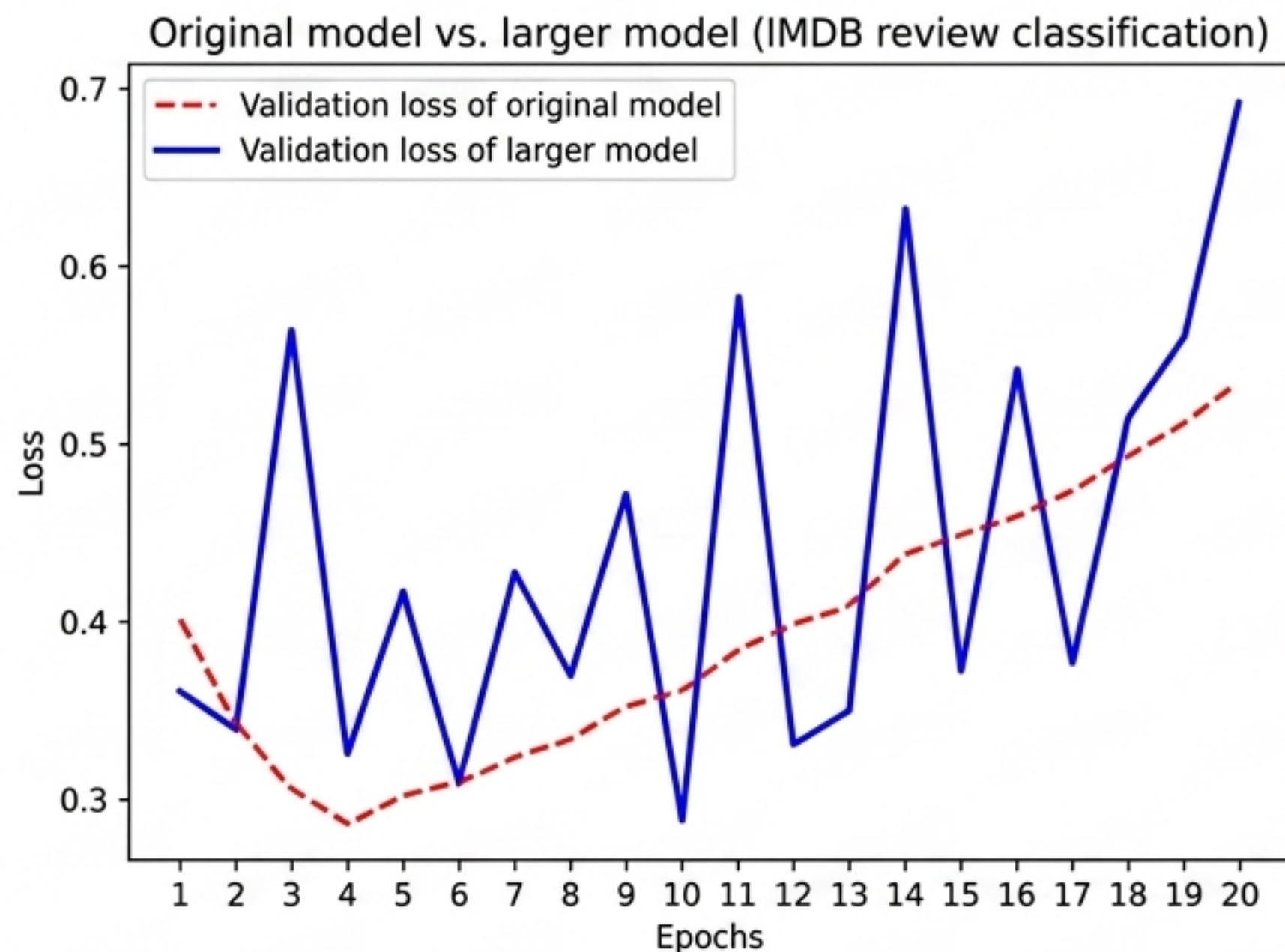
Trực quan: Mạng Gốc (16 units) vs Mạng Nhỏ (4 units)

Original model vs. smaller model (IMDB review classification)



- **Đường Dots (Chấm tròn):** Mạng gốc 16 units.
- ✕ **Đường Solid/Crosses (Đứt khúc):** Mạng nhỏ 4 units.
- ➡ **Độ trễ Overfit:** Mạng nhỏ mất thêm 6 epoch mới bắt đầu tăng Validation Loss.
- ✓ **Kết luận:** Giảm dung lượng mạng chính là kỹ thuật Regularization gốc rễ.

Trực quan: Mạng Gốc (16 units) vs Mạng Lớn (512 units)



- **Đường Solid/Crosses:** Mạng khổng lồ 512 units.
- **Hiện tượng:** Rơi vào tình trạng "Ghi nhớ thần tốc".
- **Hậu quả:** Overfit dữ dội ngay từ epoch đầu tiên. Biên độ Loss bị nhiễu cực lớn.

Phạt Trọng Số (Weight Regularization) L1 & L2

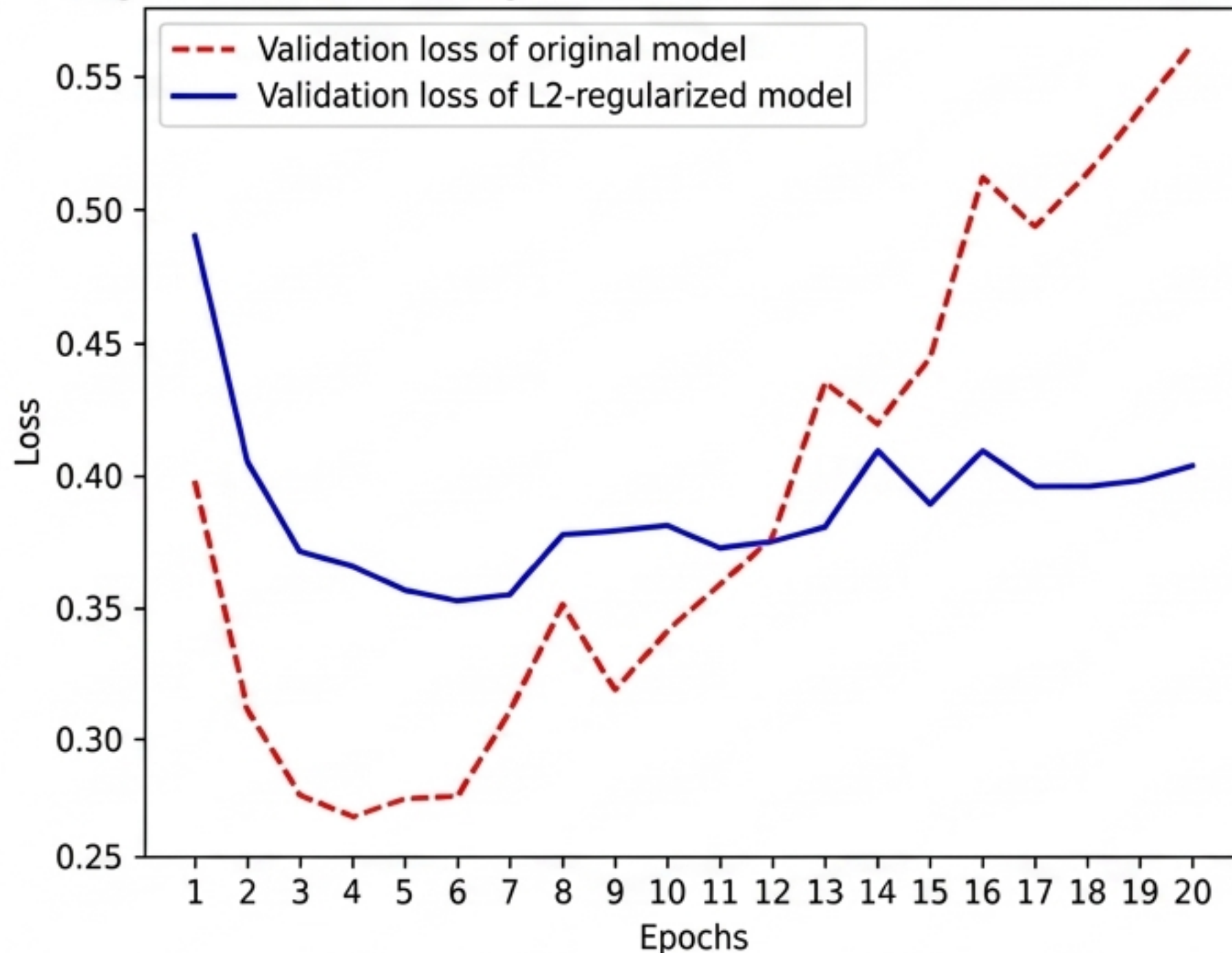


L1 Regularization	Thêm mức phạt tỷ lệ với trị tuyệt đối $ W $. Ép trọng số về 0 (Tạo sự thưa thớt - Sparsity).
L2 Regularization / Weight Decay	Thêm mức phạt tỷ lệ với bình phương W^2 . Ép trọng số tiệm cận 0.

```
layers.Dense(16, kernel_regularizer=l2(0.002))
```

Hiệu quả của Phạt trọng số L2

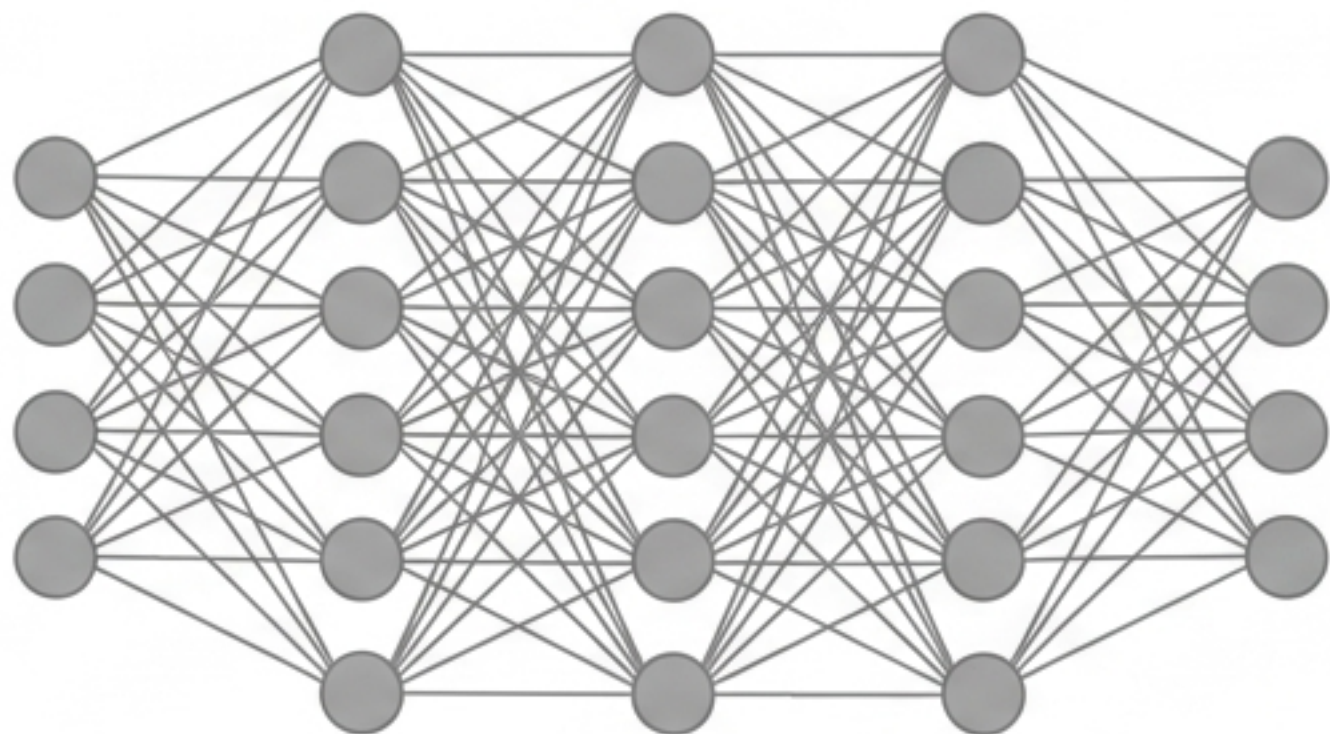
Original model vs. L2-regularized model (IMDB review classification)



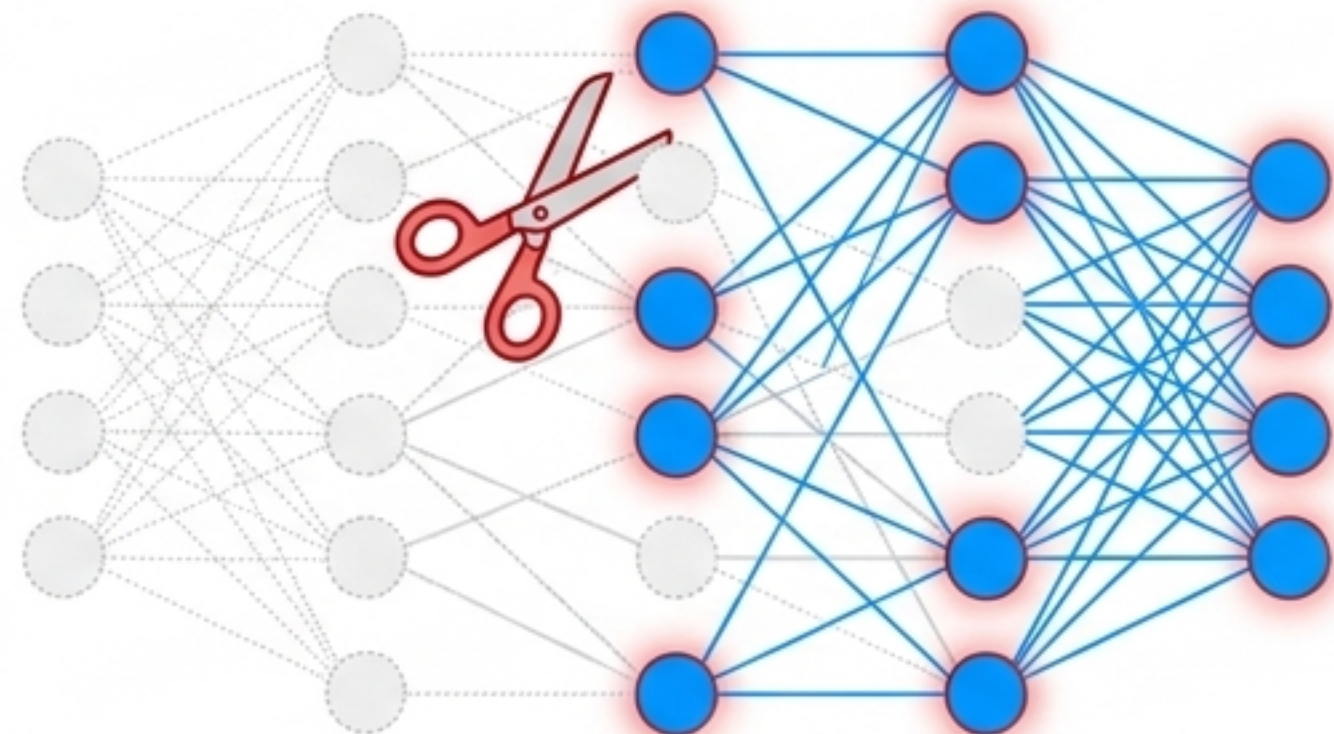
- **Cơ chế hoạt động:** Khoản phạt $0.002 \times W^2$ chỉ cộng vào hàm **Loss** lúc Huấn luyện. Nó tắt hoàn toàn lúc Kiểm tra.
- **Hiệu quả (Đường Crosses):** Duy trì Validation Loss ở mức an toàn, kháng **Overfit** lâu hơn đáng kể so với mạng gốc.
- **Ứng dụng:** Hoạt động tốt nhất trên các mô hình có **kích cỡ bé và vừa**.

Kỹ thuật Dropout (Ngắt Nơ-ron Ngẫu nhiên)

Mạng Gốc



Với Dropout



- **Phát minh:** Kỹ thuật cách mạng do Geoffrey Hinton giới thiệu.
- **Cơ chế:** Tại mỗi bước tối ưu, đột ngột vô hiệu hóa ngẫu nhiên (ép về 0) một tỷ lệ nơ-ron (thường $0.2 \sim 0.5$).
- **Mục đích:** Ngăn chặn "đồng thích nghi" (Co-adaptation). Nơ-ron buộc phải tự hình thành đặc trưng độc lập, không ỷ lại nơ-ron khác.

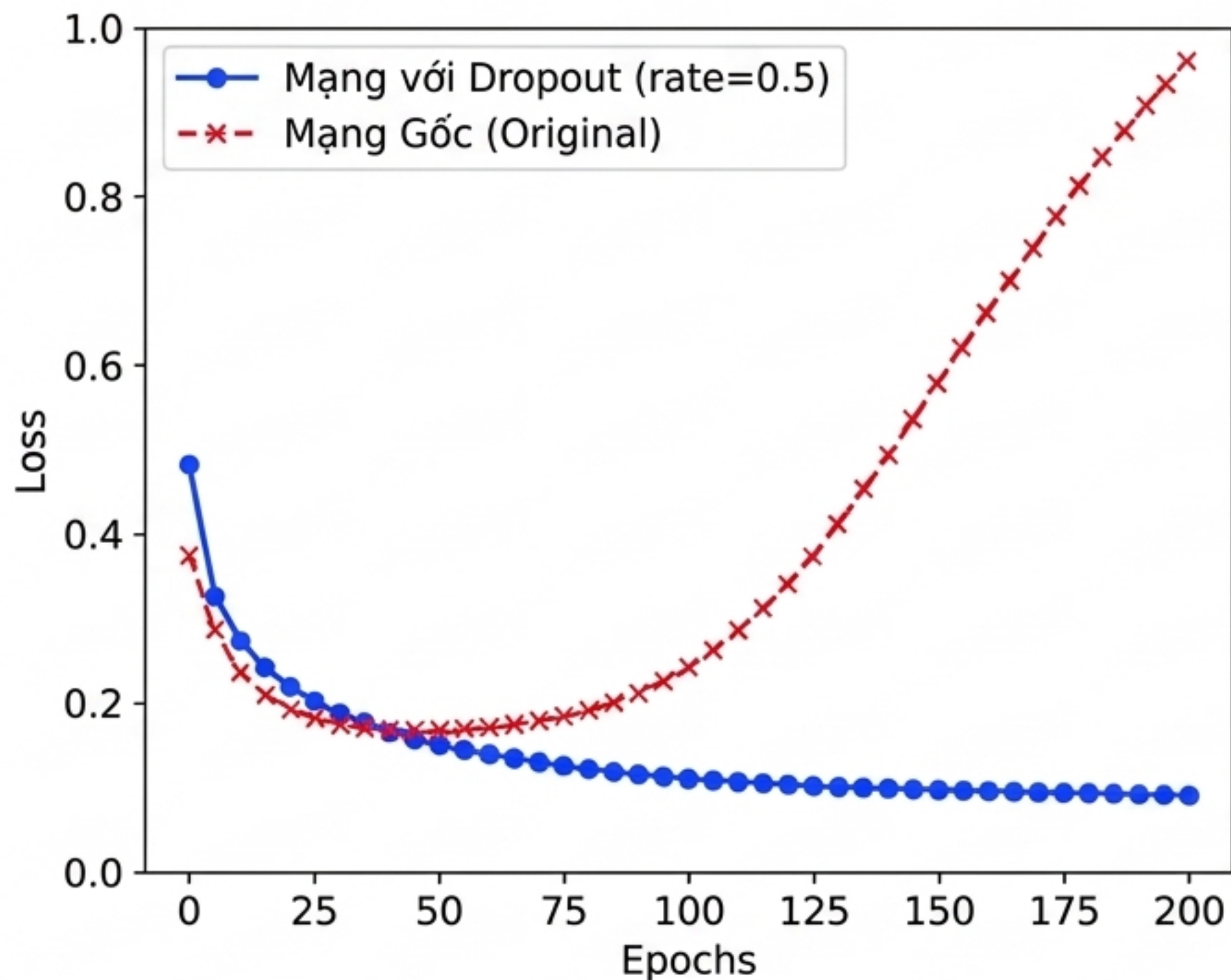
Cơ chế cân bằng Toán học của Dropout



- **Vấn đề:** Lúc Training ngắt 50% (rate = 0.5). Lúc Testing chạy full 100%. Tổng lực (Activation) tăng gấp đôi sẽ làm vỡ hàm Loss!
- **Giải pháp (Keras):** Tại bước Training, các nơ-ron sống sót tự động được Scale Up (nhân cho $1 / (1 - 0.5) = 2$).

```
layers.Dropout(0.5) # Nhét ngay sau lớp Dense
```

Hiệu quả thực tế của Dropout



- Mạng sử dụng **Dropout rate = 0.5** (Đường Solid/Crosses).
- **Kết quả:** Kháng cự lại sự **Overfit** vô cùng kiên cường.
- **Tầm quan trọng:** Đây là kỹ thuật **Regularization** hiệu quả bậc nhất, được áp dụng gần như mặc định cho các mô hình **Deep Learning** cực sâu.

Lựa chọn thay thế: Kỹ thuật Tính năng (Feature Engineering)

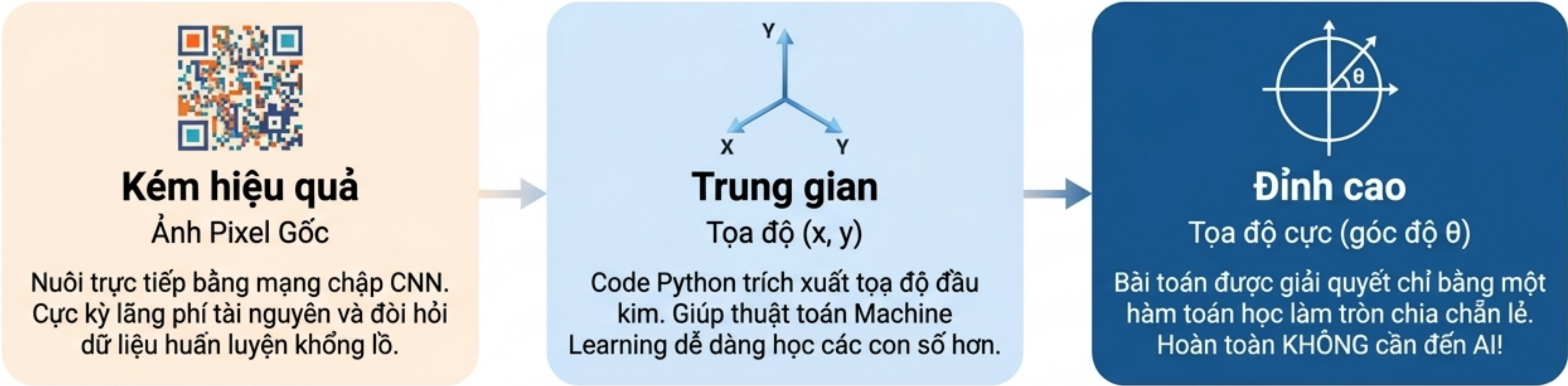


- **Định nghĩa:** Sử dụng kiến thức chuyên gia (**Domain Knowledge**) biến đổi dữ liệu đầu vào thân thiện hơn với thuật toán.
- **Lợi ích:** Dù DL có thể tự trích xuất tính năng ẩn, nhưng với dữ liệu nhỏ/đặc thù, thiết kế tốt giúp mô hình: Tốn ít tài nguyên, chạy nhanh hơn, chính xác hơn.
- **Bài toán kinh điển:** Nhận diện thời gian trên đồng hồ kim. Trích xuất góc kim đồng hồ trực tiếp thay vì bắt mạng nơ-ron tự học từ pixel thô.

Sức mạnh của Feature Engineering: Bài toán đọc Đồng hồ

Raw data: pixel grid		
Better features: clock hands' coordinates	{x1: 0.7, y1: 0.7} {x2: 0.5, y2: 0.0}	{x1: 0.0, y1: 1.0} {x2: -0.38, y2: 0.32}
Even better features: angles of clock hands	theta1: 45 theta2: 0	theta1: 90 theta2: 140

Hình 5.18: Feature Engineering cho bài toán đọc thời gian



Giải mã Không gian Dữ liệu: Giả thuyết Đa tạp (Manifold Hypothesis)



Sự vô tận của Không gian

Ảnh màu 256×256 RGB tạo ra không gian hàng tỷ tỷ chiều (nhiều hơn số nguyên tử trong vũ trụ).

Sự thật: Chọn ngẫu nhiên 1 tập pixel bất kỳ, xác suất tạo ra hình dạng hợp lý bằng 0.



Giả thuyết Manifold

Dữ liệu thực (khuôn mặt, chữ số MNIST) uốn lượn gọn gàng trên các không gian con cấu trúc cao gọi là **Đa tạp** (Manifold).

Bản chất: Học sâu là quá trình vuốt phẳng (uncrumpling) các đa tạp rối rắm này.

Nội suy Đa tạp (Manifold) bám chặt dọc theo mặt phẳng cong
→ Chuyển tiếp siêu mượt.



Manifold interpolation
(intermediate point on the latent manifold)



Linear interpolation
(average in the encoding space)

Nội suy Tuyến tính (Linear) tạo ra điểm văng ra ngoài không gian chữ số
→ Hình vô nghĩa.

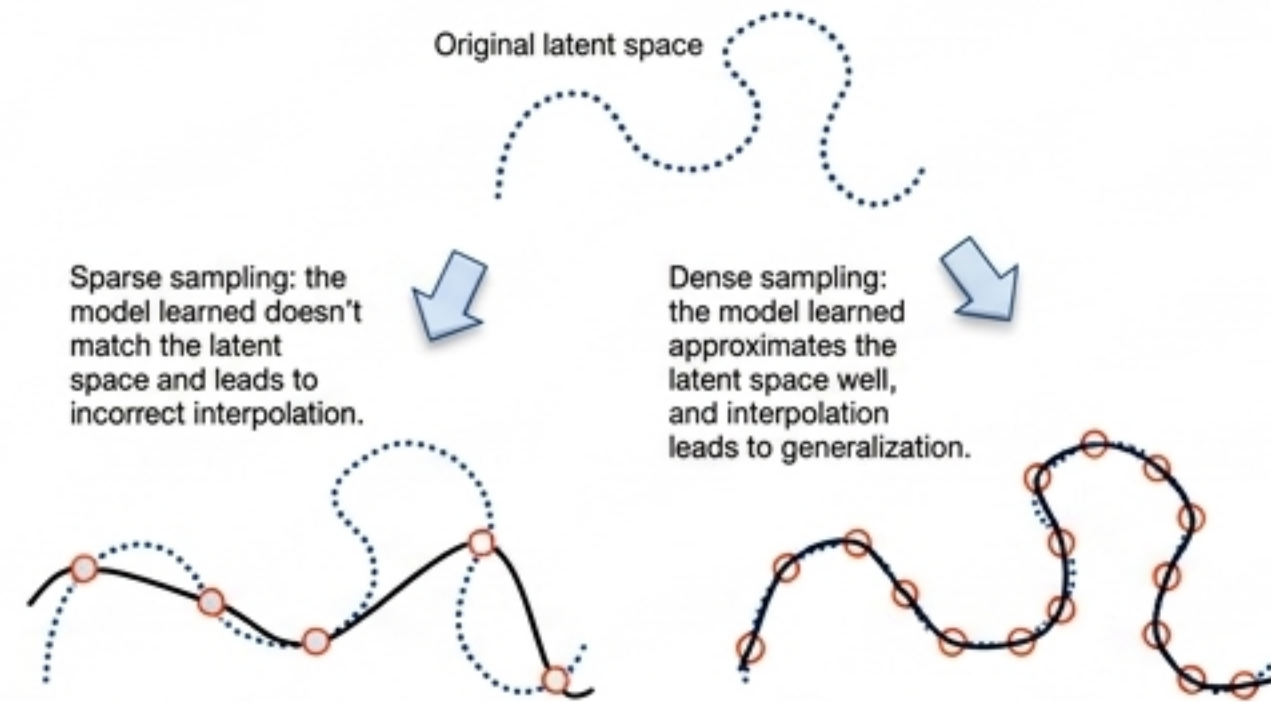
Điều kiện Cần: Mật độ Lấy mẫu (Dense Sampling) & Không gian Ẩn



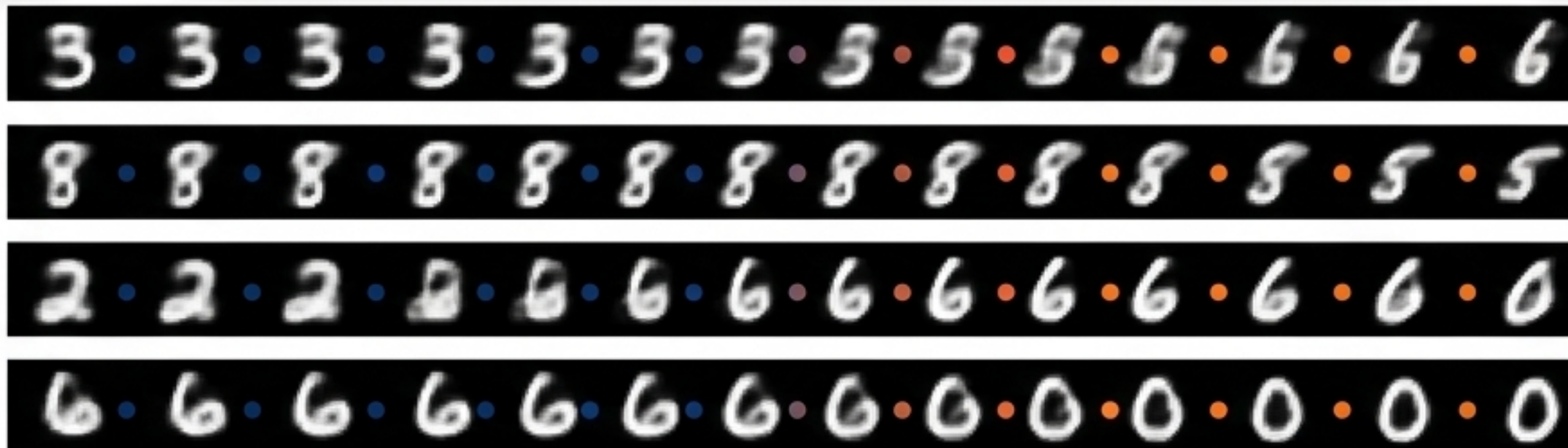
Học sâu bản chất là khớp đường cong. Để học trọn vẹn Đa tạp, lượng dữ liệu khổng lồ lấp đầy bề mặt.



Thuật toán tối tân không bù đắp được dữ liệu thưa thớt.



Không gian ẩn (Latent space) tự động gom cụm các chữ số theo từng đảo (cluster) giống nhau.



Cốt lõi Học Máy & Mối đe dọa Overfitting (Tổng kết Phần 1)



Cốt lõi Học Máy

Duy trì ranh giới mỏng manh.

Cực đại hóa **Optimization** (trên tập Train) nhưng **KHÔNG** phá hủy **Generalization** (trên tập Test).



Kẻ thù Overfitting

Luôn rình rập **phá hoại** mô hình do:

1. **Dữ liệu chứa nhiễu.**
2. **Kiến trúc mạng quá dư thừa tham số.**



Kỷ luật Đánh giá

Bắt buộc áp dụng **đúng** **Hold-out** hoặc **K-Fold Validation**.

Tuyệt đối **KHÔNG** Tuning **siêu tham số** trên tập **Test**.

Kho Vũ khí Khắc phục Overfitting

(Tổng kết Phần 2)



Xử lý Dữ liệu

Thêm dữ liệu gốc,
Data
Augmentation,
hoặc Feature
Engineering.



Thu nhỏ Mô hình

Reduce Network
Capacity (Giảm độ
phức tạp của
mạng).



Phạt Trọng số

Sử dụng L2
Regularization để
ép trọng số nhỏ
lại.



Nhiều phân tách

Dropout. Chống
hiện tượng đồng
thích nghi
(Co-adaptation)
của nơ-ron.



Ngừng sớm

Early Stopping.
Dừng huấn luyện
trước khi mô hình
bắt đầu học vẹt.